

Pengembangan Sistem Terintegrasi Berbasis Microservices System E Commers

Mata Kuliah: Integrasi Aplikasi Enterprise
Dosen Pengampu: Purnama Anaking, S.Kom., M.Kom.



Sistem Informasi Telkom University Surabaya

Disusun Oleh:

Aulia Rahman Ramaadhan	102062400008
Muhammad Badillah Fawaqih	102062400035
Muhammad Brilliant Agung	102062400126
Wicaksono	
Ahfiya Zukhrotunnabila	102062400135

**S1 SISTEM INFORMASI
FAKULTAS REKAYASA INDUSTRI T
ELKOM UNIVERSITY SURABAYA
TAHUN AKADEMIK 2026**

BAB 1 — DESKRIPSI PROJECT

1.1. Latar Belakang

Pertumbuhan transaksi digital yang terus meningkat menuntut platform e-commerce untuk memiliki sistem backend yang tidak hanya andal, tetapi juga modular dan mudah dikembangkan. Arsitektur monolitik, yang menempatkan seluruh logika bisnis dalam satu basis kode dan satu basis data, semakin sulit diandalkan ketika kompleksitas dan volume transaksi bertambah, karena perubahan kecil pada satu fitur dapat berisiko mengganggu fitur lain yang sebenarnya tidak berkaitan.

Sebagai alternatif, arsitektur microservices menawarkan pendekatan di mana setiap fungsi bisnis utama dipecah menjadi layanan independen yang memiliki siklus pengembangan, deployment, dan database sendiri. Pendekatan ini sangat relevan diterapkan pada domain e-commerce, di mana proses bisnis seperti manajemen pengguna, manajemen produk dan stok, pemrosesan pesanan, serta pengiriman notifikasi kepada pelanggan pada dasarnya dapat berjalan sebagai unit-unit yang terpisah namun tetap saling terhubung.

Dalam konteks mata kuliah [Nama Mata Kuliah], kelompok kami membangun sebuah sistem e-commerce sederhana berbasis microservices untuk mensimulasikan alur bisnis pemesanan produk secara end-to-end. Sistem ini dirancang untuk membuktikan bahwa empat layanan yang berjalan secara independen dapat saling berkomunikasi, baik secara sinkron maupun asinkron, untuk menghasilkan satu alur transaksi yang utuh dan konsisten.

1.2. Tujuan

Tujuan dari pengerjaan project ini adalah:

1. Membangun sistem e-commerce yang modular dan terintegrasi menggunakan arsitektur microservices.
2. Mengimplementasikan Docker untuk deployment berbasis container sehingga setiap layanan dapat berjalan secara terisolasi dan independen.
3. Menerapkan RESTful API sebagai layer komunikasi sinkron antar layanan.
4. Menerapkan Redis Queue sebagai mekanisme komunikasi asinkron untuk proses yang tidak memerlukan respons langsung, khususnya pembaruan stok produk.
5. Mengimplementasikan GraphQL menggunakan Lighthouse pada Product Service serta Hasura sebagai GraphQL gateway otomatis.

1.3. Nama Project

System E-Commers ini adalah sistem terintegrasi berbasis microservices yang dibangun untuk mensimulasikan proses bisnis e-commerce, mulai dari manajemen pengguna dan produk hingga pemesanan dan notifikasi. Sistem ini terdiri dari empat layanan utama yang masing-masing memiliki tanggung jawab berbeda namun saling terhubung satu sama lain.

1.4. Teknologi yang Digunakan

Teknologi	Fungsi
Laravel 13	Framework backend tiap service
Docker	Containerisasi tiap service
MySQL	Database tiap service
PostgreSQL	Database Hasura
Redis	Message broker untuk komunikasi asinkron (queue)
GraphQL Lighthouse	GraphQL manual di Product Service
Hasura	GraphQL otomatis melalui PostgreSQL
RESTful API	Komunikasi sinkron antar service
Nginx	Web server tiap service
PHP 8.4	Runtime bahasa pemrograman
Composer	Package manager PHP
Postman	Dokumentasi dan pengujian API

BAB 2 — DESKRIPSI ARSITEKTUR SISTEM

2.1. Gambaran Umum Arsitektur

Sistem E-COMMERCE CIHUY dibangun menggunakan arsitektur microservices, di mana setiap layanan berjalan secara independen di dalam container Docker masing-masing dan memiliki database sendiri (database per service). Pendekatan ini dipilih agar setiap layanan dapat dikembangkan, di-deploy, dan diskalakan secara terpisah tanpa saling mengganggu layanan lain.

Seluruh layanan terhubung dalam satu jaringan Docker bernama laravel-net, sehingga antar container dapat saling berkomunikasi menggunakan nama service sebagai hostname, tanpa perlu mengetahui alamat IP secara eksplisit.

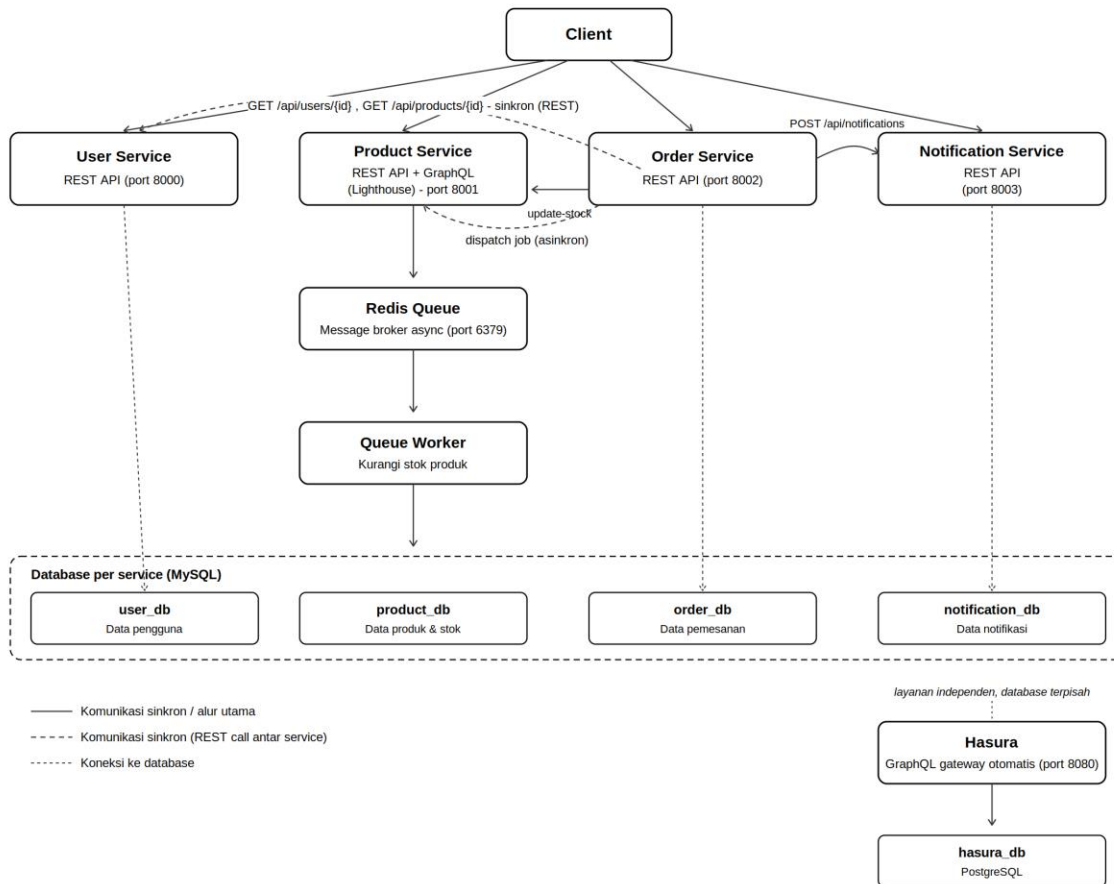
Sistem ini menggabungkan dua pola komunikasi sekaligus, yaitu komunikasi sinkron menggunakan RESTful API untuk kebutuhan yang memerlukan respons langsung, dan komunikasi asinkron menggunakan Redis Queue untuk proses yang dapat dijalankan di latar belakang tanpa membuat client menunggu, khususnya pada proses pengurangan stok produk.

2.2. Daftar Layanan Service

Service	Port	Database	Fungsi
User Service	8000	user_db (MySQL)	Manajemen data pengguna / pelanggan
Product Service	8001	product_db (MySQL)	Manajemen data produk & stok
Order Service	8002	order_db (MySQL)	Manajemen pemesanan
Notification Service	8003	notification_db (MySQL)	Manajemen notifikasi
Hasura	8080	hasura_db (PostgreSQL)	GraphQL gateway otomatis
Redis	6379	-	Message broker komunikasi asinkron

2.3. Diagram Arsitektur

[Sisipkan diagram arsitektur sistem Anda di sini, menggambarkan client, keempat service, Redis Queue, Queue Worker, database masing-masing service, serta Hasura dan PostgreSQL-nya — mengikuti pola komunikasi pada bagian 2.4.]



Gambar 2.1 Diagram Arsitektur Sistem E-COMMERCE CIHUY

2.4. Komunikasi Antar Service

2.4.1. Komunikasi Sinkron (RESTful API)

Komunikasi sinkron terjadi ketika satu service memanggil service lain dan menunggu respons sebelum melanjutkan proses. Pola ini digunakan ketika data dari service lain dibutuhkan secara langsung, misalnya saat Order Service membutuhkan data pengguna dan data produk untuk menyusun detail pesanan.

2.4.2. Komunikasi Asinkron (Redis Queue)

Komunikasi asinkron digunakan agar service pengirim tidak perlu menunggu proses selesai di service penerima. Pola ini diterapkan pada proses pengurangan stok produk ketika order baru dibuat, sehingga client dapat menerima respons order tanpa harus menunggu proses pembaruan stok selesai diproses.

2.4.3. GraphQL — Lighthouse dan Hasura

Selain RESTful API, sistem E-COMMERCE CIHUY juga menyediakan akses data melalui GraphQL dengan dua pendekatan berbeda:

- GraphQL Lighthouse diimplementasikan secara manual di Product Service menggunakan package lighthouse. Skema GraphQL didefinisikan dalam file `schema.graphql` dan mendukung query serta mutation untuk entitas Product (create, read, update, delete).
- GraphQL Hasura digunakan sebagai GraphQL gateway otomatis yang terhubung ke database PostgreSQL terpisah (`hasura_db`). Hasura secara otomatis menghasilkan skema GraphQL lengkap (query dan mutation) hanya dengan membuat tabel melalui Hasura Console, tanpa perlu menulis kode backend tambahan.

2.5. Docker Network

Seluruh container dalam sistem E-COMMERCE CIHUY terhubung melalui satu jaringan Docker bernama `laravel-net` dengan driver `bridge`. Jaringan ini memungkinkan setiap container saling berkomunikasi menggunakan nama container sebagai `hostname` internal.

Service	Hostname Internal
User Service	<code>user-service-nginx</code>
Product Service	<code>product-service-nginx</code>
Order Service	<code>order-service-nginx</code>
Notification Service	<code>notification-service-nginx</code>
Redis	<code>redis</code>
Hasura	<code>hasura</code>
PostgreSQL (Hasura)	<code>hasura-db</code>

BAB 3 — DOKUMENTASI API

3.1. Postman Collection

Dokumentasi seluruh endpoint API, baik RESTful API maupun GraphQL, didokumentasikan menggunakan Postman dan dipublikasikan agar dapat diakses secara publik.

LINKKKK

3.2. Struktur Collection Postman

- E-COMMERCE CIHUY - API Collection
 - a) User Service REST
 - b) Product Service REST
 - c) GraphQL Lighthouse (Product Service)
 - d) GraphQL Hasura
 - e) Order Service REST
 - f) Notification Service REST

Total: [jumlah] endpoint terdokumentasi

3.3. Link Repositori

LINKKKKK

BAB 4 — PENJELASAN FITUR DAN FLOW

4.1. Fitur User Service

User Service bertanggung jawab mengelola data pengguna / pelanggan pada sistem E-COMMERCE CIHUY.

Endpoint RESTful API:

Method	Endpoint	Fungsi
GET	/api/users	Mengambil seluruh data pengguna
POST	/api/users	Membuat pengguna baru
GET	/api/users/{id}	Mengambil data pengguna berdasarkan ID
PUT	/api/users/{id}	Memperbarui data pengguna
DELETE	/api/users/{id}	Menghapus data pengguna

4.2. Fitur Product Service

Product Service mengelola data produk dan stok barang. Selain RESTful API, layanan ini juga menyediakan akses GraphQL secara manual menggunakan Lighthouse. Fitur utamanya adalah pembaruan stok yang diproses secara asinkron menggunakan Redis Queue, sehingga proses pengurangan stok tidak menghambat respons ke client.

Endpoint RESTful API:

Method	Endpoint	Fungsi
GET	/api/products	Mengambil seluruh data produk
POST	/api/products	Membuat produk baru
GET	/api/products/{id}	Mengambil produk berdasarkan ID
PUT	/api/products/{id}	Memperbarui data produk
POST	/api/products/{id}/update-stock	Memperbarui stok secara asinkron melalui Redis Queue
DELETE	/api/products/{id}	Menghapus produk

Endpoint GraphQL Lighthouse (/graphql):

Tipe	Operasi
Query	products — mengambil semua produk
Query	product(id) — mengambil produk berdasarkan ID
Mutation	createProduct — membuat produk baru
Mutation	updateProduct — memperbaiki produk

Mutation	deleteProduct — menghapus produk
----------	----------------------------------

4.3. Fitur Order Service

Order Service menjadi titik pusat alur bisnis pemesanan produk. Layanan ini melakukan komunikasi sinkron ke User Service dan Product Service untuk mengambil data lengkap saat menampilkan detail order, serta memicu komunikasi asinkron ke Product Service untuk pembaruan stok.

Endpoint RESTful API:

Method	Endpoint	Fungsi
GET	/api/orders	Mengambil seluruh data order
POST	/api/orders	Membuat order baru
GET	/api/orders/{id}	Mengambil detail order beserta data user dan produk
PUT	/api/orders/{id}	Memperbarui status order
DELETE	/api/orders/{id}	Menghapus order
GET	/api/orders/user/{userId}	Mengambil seluruh order milik user tertentu

4.4. Fitur Notification Service

Notification Service mengelola pencatatan notifikasi yang dikirimkan kepada pengguna, misalnya saat order berhasil dibuat atau status order berubah.

Endpoint RESTful API:

Method	Endpoint	Fungsi
GET	/api/notifications	Mengambil seluruh notifikasi
POST	/api/notifications	Membuat notifikasi baru
GET	/api/notifications/{id}	Mengambil notifikasi berdasarkan ID
DELETE	/api/notifications/{id}	Menghapus notifikasi
GET	/api/notifications/recipient/{recipient}	Mengambil notifikasi berdasarkan penerima

4.5. Fitur GraphQL Hasura

Hasura menyediakan akses GraphQL otomatis ke database PostgreSQL terpisah (hasura_db), digunakan untuk mendemonstrasikan kemampuan GraphQL gateway tanpa menulis kode backend tambahan.

Endpoint: <http://localhost:8080/v1/graphql>

Tipe	Operasi
------	---------

Query	— mengambil seluruh data
Mutation	insert_[nama_tabel]_one — menambah data baru
Mutation	delete_[nama_tabel]_by_pk — menghapus data

4.6. Flow Fitur Utama

Berikut adalah alur lengkap ketika seorang pelanggan membuat order baru, yang melibatkan keempat service sekaligus:

Step 1 — Persiapan Data

Pelanggan terlebih dahulu terdaftar melalui User Service, dan data produk tersedia melalui Product Service, untuk mendapatkan ID/UUID masing-masing entitas.

Step 2 — Pembuatan Order

Client mengirim POST /api/orders ke Order Service dengan menyertakan user_id, product_id, total_price, dan quantity. Order Service menyimpan data order ke database order_db.

Step 3 — Komunikasi Asinkron untuk Update Stok

Setelah order tersimpan, Order Service mengirim request ke endpoint update-stock milik Product Service. Product Service tidak langsung memproses pengurangan stok, melainkan men-dispatch job pembaruan stok ke Redis Queue. Order Service langsung membalas ke client tanpa menunggu proses ini selesai.

Step 4 — Pemrosesan Stok di Background

Queue Worker pada Product Service mengambil job dari Redis Queue dan memproses pengurangan stok produk di product_db secara asinkron.

Step 5 — Pengambilan Detail Order (Komunikasi Sinkron)

Ketika client mengambil detail order melalui GET /api/orders/{id}, Order Service melakukan pemanggilan REST API secara sinkron ke User Service (GET /api/users/{id}) dan Product Service (GET /api/products/{id}), kemudian menggabungkan ketiga data tersebut menjadi satu response lengkap.

Step 6 — Notifikasi

Sebagai pelengkap alur bisnis, Notification Service mencatat notifikasi terkait status order melalui endpoint POST /api/notifications, yang dapat diintegrasikan untuk memberi tahu pelanggan bahwa pesanan mereka berhasil diproses.

4.7. Ringkasan Pola Komunikasi yang Diimplementasikan

Pola Komunikasi	Digunakan pada	Teknologi
Sinkron	Order Service ↔ User Service & Product Service	RESTful API (Laravel HTTP Client)

Asinkron	Order/Product Service → Queue Worker	Redis Queue
GraphQL manual	Product Service	Lighthouse
GraphQL otomatis	Layanan independen via PostgreSQL	Hasura

BAB 5 — KESIMPULAN

Melalui pengerjaan tugas besar ini, kelompok kami berhasil membangun sistem e-commerce terintegrasi berbasis microservices bernama E-COMMERCE CIHUY yang terdiri dari empat layanan independen — User Service, Product Service, Order Service, dan Notification Service — masing-masing dengan database terpisah. Sistem ini membuktikan bahwa kombinasi komunikasi sinkron melalui RESTful API dan komunikasi asinkron melalui Redis Queue dapat berjalan berdampingan untuk menghasilkan satu alur transaksi yang utuh dan efisien, khususnya pada proses pembuatan order dan pembaruan stok produk.

Selain itu, implementasi dua pendekatan GraphQL Lighthouse secara manual pada Product Service dan Hasura secara otomatis pada database PostgreSQL terpisah menunjukkan fleksibilitas dalam menyediakan layer komunikasi data yang sesuai dengan kebutuhan masing-masing layanan. Secara keseluruhan, project ini memberikan pemahaman praktis mengenai bagaimana arsitektur microservices, containerisasi berbasis Docker, serta kombinasi komunikasi sinkron dan asinkron dapat diterapkan untuk membangun sistem enterprise yang modular, skalabel, dan mudah dikembangkan.